1. a) Create 3 pointers called P1, P2, P3 and an integer variable X where P1 points to P2, P2 points to P3 and P3 points to X. Demonstrate how to read and retrieve the value of X using P1, P2 and P3.

```
#include <stdio.h>

int main() {

int X = 100;

// P3 points to an integer (X)

int *P3;

// P2 points to a pointer to an integer (P3)

int **P2;

// P1 points to a pointer to a pointer to an integer (P2)

int ***P1;

// Establish the pointer chain

P3 = &X; // P3 points to the address of X

P2 = &P3; // P2 points to the address of P3

P1 = &P2; // P1 points to the address of P2

// Retrieve the value of X using the pointer chain

// Dereference P1 to get P2's address, then dereference P2 to get P3's address,

// then dereference P3 to get the value of X.

int retrieved_X = ***P1;

// Print the value of X

printf("The value of X is: %d\n", X);

printf("The value of X retrieved using P1, P2, and P3 is: %d\n", retrieved_X);

return 0;

}
```

1.b Crate a pointer to an integer array, Read and display the contents using pointer and also display the contents in reverse order using pointer.

```
#include <stdio.h>

int main() {

int arr[5]; // Declare an integer array of size 5

int *ptr; // Declare an integer pointer

// Assign the base address of the array to the pointer

ptr = arr;

// Read contents into the array using the pointer

printf("Enter 5 integer elements:\n");

for (int i = 0; i < 5; i++) {

printf("Element %d: ", i + 1);

scanf("%d", (ptr + i)); // Use pointer arithmetic to access elements

}

// Display contents of the array using the pointer

printf("\nArray elements (normal order):\n");

for (int i = 0; i < 5; i++) {

printf("%d ", *(ptr + i)); // Dereference the pointer to get the value

}

printf("\n");

// Display contents of the array in reverse order using the pointer

printf("\nArray elements (reverse order):\n");

for (int i = 4; i >= 0; i--) { // Start from the last element and go backwards

printf("%d ", *(ptr + i));

}

printf("\n");

return 0; }
```

1.C. Create a pointer to an integer array and pass this as parameter to a function called Reverse which reverses the array contents using pointer. Write different functions called Read (P) to read the array elements using pointer P and Display (P) to display the array contents using P before and after reverse

```
#include <stdio.h>

// Function to read array elements using a pointer

void Read(int *p, int size) {

printf("Enter %d integer elements:\n", size);

for (int i = 0; i < size; i++) {

scanf("%d", (p + i)); // Read into the memory location pointed by (p + i)

}

}

// Function to display array contents using a pointer

void Display(int *p, int size) {

for (int i = 0; i < size; i++) {

printf("%d ", *(p + i)); // Dereference and print the value at (p + i)

}

printf("\n");

}

// Function to reverse the array contents using pointers

void Reverse(int *p, int size) {

int *start = p;

int *end = p + size - 1;

while (start < end) {

// Swap elements pointed by start and end

int temp = *start;

*start = *end;

*end = temp;

start++; // Move start pointer forward
```

```
end--; // Move end pointer backward
}
}
int main() {
int arr_size;
printf("Enter the size of the array: ");
scanf("%d", &arr_size);
int *arr_ptr = (int *)malloc(arr_size * sizeof(int)); // Dynamically allocate memory for the array
if (arr_ptr == NULL) {
printf("Memory allocation failed!\n");
return 1; // Indicate an error
}
// Read array elements
Read(arr_ptr, arr_size);
printf("Array before reverse: ");
Display(arr_ptr, arr_size);
// Reverse the array
Reverse(arr_ptr, arr_size);
printf("Array after reverse: ");
Display(arr_ptr, arr_size);
free(arr_ptr); // Free the dynamically allocated memory
arr_ptr = NULL; // Set the pointer to NULL after freeing
return 0;
}
```

1.d Create a pointer to two-dimensional array of integers. Write functions Read (P) and Display (P) to read and display the contents using pointer.

```
#include <stdio.h>

// Define the dimensions of the 2D array

#define ROWS 3

#define COLS 4

// Function to read contents into the 2D array using a pointer

void Read(int (*P)[COLS]) {

printf("Enter %d x %d integer values:\n", ROWS, COLS);

for (int i = 0; i < ROWS; i++) {

for (int j = 0; j < COLS; j++) {

// Access elements using pointer arithmetic

scanf("%d", (*(P + i) + j));

}

}

}

// Function to display contents of the 2D array using a pointer

void Display(int (*P)[COLS]) {

printf("\nContents of the 2D array:\n");

for (int i = 0; i < ROWS; i++) {

for (int j = 0; j < COLS; j++) {

// Access elements using pointer arithmetic

printf("%d\t", *(*(P + i) + j));

}

printf("\n");

}

}

int main() {
```

```
// Declare a 2D array
int arr[ROWS][COLS];
// Declare a pointer to a 2D array (specifically, a pointer to an array of COLS integers)
int (*ptr_to_array)[COLS];
// Assign the address of the 2D array to the pointer
ptr_to_array = arr;
// Call the Read function to populate the array using the pointer
Read(ptr_to_array);
// Call the Display function to print the array contents using the pointer
Display(ptr_to_array);
return 0;
}
```

1.e Create two pointers called PS1 and PS2 to two strings, Read the contents using pointers and also write a function StrCat (PS1, PS2) which concatenates second string to the end of first string using pointers PS1 and PS2.

```
#include <stdio.h>

// Function to concatenate the second string to the end of the first string

void StrCat(char *PS1, char *PS2) {

// Move PS1 to the end of the first string (null terminator)

while (*PS1 != '\0') {

PS1++;

}

// Copy characters from PS2 to the end of PS1

while (*PS2 != '\0') {

*PS1 = *PS2;

PS1++;

PS2++;

}

// Add the null terminator to the end of the concatenated string

*PS1 = '\0';

}

int main() {

// Declare two character arrays (strings)

char str1[100]; // Ensure enough space for concatenation

char str2[50];

// Declare two pointers to characters (strings)

char *PS1 = str1;

char *PS2 = str2;

// Read contents into the strings using pointers

printf("Enter the first string: ");

scanf("%s", PS1); // scanf reads until whitespace
```

```
printf("Enter the second string: ");
scanf("%s", PS2);
// Print the original strings using pointers
printf("\nOriginal String 1: %s\n", PS1);
printf("Original String 2: %s\n", PS2);
// Concatenate PS2 to PS1 using the custom StrCat function
StrCat(PS1, PS2);
// Print the concatenated string
printf("\nConcatenated String (PS1): %s\n", PS1);
return 0;
}
```